

Sistemas Distribuídos — Semana 05

RPC, RMI e Comunicação por Eventos · Aulas 9 (01/09) e 10 (03/09) · Prof. Leandro Borges

RPC: Remote Procedure Call

A ideia central do RPC é esconder a complexidade da rede: o programador chama uma função remota como se fosse uma chamada de função comum, local. Essa transparência é o que torna o RPC tão atraente — em vez de lidar diretamente com sockets, o desenvolvedor escreve algo como `client.somar(2, 3)`, e toda a comunicação de rede acontece por trás dos panos.

Isso é possível graças aos stubs de cliente e servidor: código gerado automaticamente que empacota (marshalling) os parâmetros da chamada e desempacota o valor de retorno. É importante ter em mente, porém, que essa transparência nunca é total: falhas de rede, latência e diferenças de representação de tipos de dados entre máquinas fazem o RPC se comportar, em alguns aspectos, de forma diferente de uma chamada local — mas a abstração ainda economiza muito trabalho do desenvolvedor.

Como funciona uma chamada RPC

O fluxo típico de uma chamada RPC segue quatro passos. Primeiro, o cliente faz uma chamada local aparente, invocando o stub como se fosse uma função comum. Segundo, ocorre o empacotamento (marshalling): o stub do cliente serializa os parâmetros da chamada em uma mensagem que pode trafegar pela rede. Terceiro, essa mensagem é enviada e a execução remota acontece: ela viaja até o servidor, onde o stub correspondente a desempacota e chama a função real. Quarto, o retorno: o resultado é empacotado no servidor, enviado de volta pela rede e entregue ao código cliente como se fosse um retorno de função comum.

RPC vs. RMI

O RPC segue um paradigma procedural: o que se invoca é uma função ou procedimento remoto, como em tecnologias como gRPC ou XML-RPC. O RMI (Remote Method Invocation) traz essa mesma ideia para o paradigma orientado a objetos: o que se invoca é um método de um objeto remoto, sendo o Java RMI o exemplo mais conhecido dessa tecnologia.

O RMI carrega consigo conceitos próprios do mundo de objetos distribuídos, como referências remotas a objetos, coleta de lixo distribuída (para saber quando um objeto remoto não é mais referenciado por ninguém) e a decisão entre passar objetos por valor (uma cópia) ou por referência (um ponteiro remoto) entre as chamadas.

Serviço de grupo e comunicação por eventos

O serviço de grupo permite enviar uma mensagem para todo um grupo de processos de uma só vez (multicast), em vez de repetir o envio individualmente para cada um. Isso facilita bastante a replicação e a coordenação de grupos de servidores, um tema que vamos aprofundar mais adiante no semestre, quando tratarmos de replicação e consistência.

Já a comunicação orientada a eventos, também chamada de publish/subscribe, funciona de forma diferente: processos publicam eventos em um tópico, e quem tiver interesse se inscreve (subscribe) para recebê-los. A grande vantagem desse modelo é o desacoplamento entre emissor e

receptor — quem publica um evento não precisa saber quem, se é que alguém, vai efetivamente consumi-lo. Esse padrão é a base de sistemas de mensageria amplamente usados na indústria, como Kafka e RabbitMQ.

Referências

COULOURIS, G. et al. Sistemas Distribuídos: Conceitos e Projeto. 5. ed. Porto Alegre: Bookman.

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.